R Data Structures

Seminar 3 | BUSS256 2027

This tutorial starts after your R and RStudio setup is complete. You will use a script to create small objects, inspect their results, and learn how R organizes business data.

1. Start in your project

Use the workflow below so your code stays with your course files.

1. Open your BUSS256 RStudio Project.
2. Create or open a Seminar 3 R script. A useful name is seminar3.R.
3. Type code in the script.
4. Run code and inspect results in the Console.

In this handout, CODE is what you type in your script. EXPECTED OUTPUT is what you should see after you run the code. Do not type output or a Console prompt into your script.

2. Quick vector review

A vector is a sequence of values. The next example creates a vector named x, shows it, and calculates its arithmetic mean.

CODE

```
x <- c(1, 2, 3)
x
mean(x)
```

EXPECTED OUTPUT

```
[1] 1 2 3
[1] 2
```

PREDICT	RUN	COMPARE
Before you run mean(x), what result do you expect?	Run the code in your script.	Was the result what you expected?

The assignment operator <- stores a result in a name. Here, x stores the three values. The mean function uses x as its input and returns 2.

3. Why data structures matter

Business data often has rows and columns: customers, products, transactions, and quantities. R uses different structures to organize information. A vector is one sequence of values. A matrix and a data frame both use rows and columns.

4. Matrix

A matrix organizes values in rows and columns. All values in a matrix have the same data type. Start with six values, then arrange them into two rows and three columns.

CODE

```
a <- seq(1, 6)
matrix(a, 2, 3)
```

EXPECTED OUTPUT

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

R fills the matrix down the first column, then down the next column. The labels [,1], [,2], and [,3] identify columns.

Make a matrix shape visible

You can also give a vector dimensions. The object b has twelve values; dim(b) tells R to use three rows and four columns.

CODE

```
b <- 1:12
dim(b) <- c(3, 4)
b
```

EXPECTED OUTPUT

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

5. Access values in a matrix

Use square brackets to select values. The first number is the row; the second is the column. For example, b[2, 4] means row 2, column 4.

CODE
```
b[2, 4]
```

EXPECTED OUTPUT
```
[1] 11
```

PREDICT	RUN	COMPARE
Look at b. What value is in row 3, column 1?	Run the code in your script.	Was the result what you expected?

CODE
```
b[3, 1]
```

EXPECTED OUTPUT
```
[1] 3
```

To select a complete row or column, leave one position empty but keep the comma.

CODE
```
b[3, ]
b[, 1]
b[, 2:4]
```

EXPECTED OUTPUT
```
[1]  3  6  9 12
[1] 1 2 3
     [,1] [,2] [,3]
[1,]    4    7   10
[2,]    5    8   11
[3,]    6    9   12
```

From matrix to data frame

Matrices are useful when every value has the same kind of information. Business tables often combine IDs, names, items, and quantities. That is where a data frame is useful.

6. Data frame

A data frame is a common structure for business data. Each row is one observation or case. Each column is one variable. Unlike a matrix, different columns can contain different kinds of information.

This small sales object has three customer transactions. It is course-owned toy data, not an external dataset.

CODE

```
id <- c("C001", "C002", "C003")
name <- c("Gun", "Tom", "Bob")
item <- c("Pencil", "Paper", "Eraser")
qty <- c(3, 6, 2)
sales <- data.frame(ID = id, Name = name, Item = item, Quantity = qty)
sales
```

EXPECTED OUTPUT

```
    ID Name   Item   Quantity
1 C001 Gun    Pencil        3
2 C002 Tom    Paper         6
3 C003 Bob    Eraser        2
```

The rows are individual transactions. The columns are variables: ID, Name, Item, and Quantity. The ID and Name columns contain text; Quantity contains numbers.

One value with row and column

The same [row, column] idea works with a data frame. This asks for row 2 and column 3.

CODE

```
sales[2, 3]
```

EXPECTED OUTPUT

```
[1] "Paper"
```

7. Access data clearly

A named column with $

Use $ when you know the column name. This selects the Name column from sales.

CODE
```
sales$Name
```

EXPECTED OUTPUT
```
[1] "Gun" "Tom" "Bob"
```

A row and a column with [row, column]

Use [row, column] when you want a position in the table. This is the same syntax you used for a matrix.

CODE
```
sales[2, 3]
```

EXPECTED OUTPUT
```
[1] "Paper"
```

A simple comparison

The operator == asks whether two values are equal. The next line keeps the row where ID is C002. Notice that == compares; <- assigns a result to a name.

PREDICT	RUN	COMPARE
Which customer row do you expect this line to keep?	Run the code in your script.	Was the result what you expected?

CODE
```
sales[sales$ID == "C002", ]
```

EXPECTED OUTPUT
```
    ID Name  Item  Quantity
2 C002 Tom   Paper        6
```

Read the expression from the inside out: sales$ID selects the ID column, == "C002" identifies the matching row, and the outer brackets return that row from sales.

8. A simple function

A function performs an operation. A function has a name, takes input, and returns a result. The mean function calculates the arithmetic mean of numeric values.

CODE

```
D <- 1:10
mean(D)
```

EXPECTED OUTPUT

```
[1] 5.5
```

PREDICT	RUN	COMPARE
What result do you expect when R finds the mean of the values from 1 through 10?	Run the code in your script.	Was the result what you expected?

In mean(D), mean is the function name, D is the input, and 5.5 is the returned result. This tutorial uses functions; it does not teach how to define your own functions.

9. Conceptual check

TERM	WHAT IT IS FOR
VECTOR	A sequence of values.
MATRIX	Values organized in rows and columns.
DATA FRAME	Rows are observations or cases; columns are variables that can have different data types.
FUNCTION	A reusable operation that takes input and returns a result.

Before Seminar 3 ends, make sure you can explain which structure you would use for one sequence of values, a same-type grid, a business table, and a reusable calculation.

Next step: save your seminar3.R script inside the BUSS256 Project so you can return to these examples later.